```
    }
}
```

❶ Derives from BackgroundService to create a task that runs for the lifetime of your app

❷ Injects an IServiceProvider so you can create instances of the typed client

❸ A simple cache for exchange rates

❹ You must override ExecuteAsync to set the service's behavior.

❺ The CancellationToken passed as an argument is triggered when the application shuts down.

❻ Keeps looping until the application shuts down

❼ Creates a new instance of the typed client so that the HttpClient is short-lived

❽ Fetches the latest rates from the remote API

❾ Stores the rates in the cache

❿ Waits for 5 minutes (or for the application to shut down) before updating the cache

The `ExchangeRateCache` in listing 34.2 is a simple singleton that stores the latest rates. It must be thread-safe, as it is accessed concurrently by your API endpoint. You can see a simple implementation in the source code for this chapter.

To register your background service with the dependency injection (DI) container, use the `AddHostedService()` extension method in Program.cs, which registers the service using a singleton lifetime, as shown in the following listing.

**Listing 34.3 Registering an `IHostedService` with the DI container**

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
builder.Services.AddHttpClient<ExchangeRatesClient>();              ❶
builder.Services.AddSingleton<ExchangeRatesCache>();                ❷
builder.Services.AddHostedService<ExchangeRatesHostedService>();    ❸
```

❶ Registers the typed client as before

❷ Adds the cache object as a singleton so it is shared throughout your app

❸ Registers ExchangeRatesHostedService as an IHostedService